

Anexo A: Códigos Completos

A.1 MenuController.cs

C#

```
using UnityEngine;

public class MenuController : MonoBehaviour
{
    public GameObject painelMenuInicial;
    public GameObject painelOpcoes;
    public GameObject painelCreditos;

    public void Jogar()
    {
        Debug.Log("Jogo ainda não implementado");
    }

    public void AbrirOpcoes()
    {
        painelMenuInicial.SetActive(false);
        painelOpcoes.SetActive(true);
        painelCreditos.SetActive(false);
    }

    public void AbrirCreditos()
    {
        painelMenuInicial.SetActive(false);
        painelOpcoes.SetActive(false);
        painelCreditos.SetActive(true);
    }

    public void VoltarMenu()
    {
        painelMenuInicial.SetActive(true);
        painelOpcoes.SetActive(false);
        painelCreditos.SetActive(false);
    }
}
```

A.2 PlayerSettings.cs

C#

```
using UnityEngine;
using UnityEngine.UI;

[RequireComponent(typeof(CharacterController))]
public class PlayerSettings : MonoBehaviour
{
    [Header("Movimento")]
    [SerializeField] private float velocidadeAndar = 5f;
    [SerializeField] private float velocidadeCorrer = 10f;
    [SerializeField] private float gravidade = -9.81f;

    [Header("Stamina")]
    [SerializeField] private float staminaMaxima = 100f;
    [SerializeField] private float staminaGastoPorSegundo = 20f;
    [SerializeField] private float staminaRecuperacaoPorSegundo = 15f;
    [SerializeField] private float delayParaRecuperar = 2f;

    [Header("UI (Opcional)")]
    [SerializeField] private Slider barraStamina;
    [SerializeField] private Image preenchimentoStamina;
    [SerializeField] private Color corStaminaCheia = new Color(0.2f, 0.85f, 0.4f);
    [SerializeField] private Color corStaminaBaixa = new Color(0.9f, 0.2f, 0.2f);

    // Componentes
    private CharacterController controller;

    // Estado de movimento
    private Vector3 velocidade;
    private bool estaNoChao;

    // Estado de stamina
    private float staminaAtual;
    private float timerRecuperacao;
    private bool podeRecuperar;
    private bool staminaEsgotada;

    private void Awake()
    {
        controller = GetComponent<CharacterController>();
    }

    void Start()
    {
        staminaAtual = staminaMaxima;
        podeRecuperar = true;

        if (barraStamina != null)
```

```

    {
        barraStamina.maxValue = staminaMaxima;
        barraStamina.value = staminaAtual;
    }
}

void Update()
{
    TratarMovimento();
    TratarStamina();
    AtualizarUI();
}

private void TratarMovimento()
{
    estaNoChao = controller.isGrounded;
    if (estaNoChao && velocidade.y < 0)
        velocidade.y = -2f;

    float horizontal = Input.GetAxis("Horizontal");
    float vertical = Input.GetAxis("Vertical");

    Vector3 direcao = transform.right * horizontal + transform.forward * ve

    bool tentandoCorrer = Input.GetKey(KeyCode.LeftShift) || Input.GetKey(K
    bool estaMovendo = direcao.sqrMagnitude > 0.01f;
    bool correndo = tentandoCorrer && estaMovendo && !staminaEsgotada;

    float velocidadeAtual = correndo ? velocidadeCorrer : velocidadeAndar;

    controller.Move(direcao * velocidadeAtual * Time.deltaTime);

    velocidade.y += gravidade * Time.deltaTime;
    controller.Move(velocidade * Time.deltaTime);
}

private void TratarStamina()
{
    bool tentandoCorrer = Input.GetKey(KeyCode.LeftShift) || Input.GetKey(K
    float horizontal = Input.GetAxis("Horizontal");
    float vertical = Input.GetAxis("Vertical");
    bool estaMovendo = new Vector2(horizontal, vertical).sqrMagnitude > 0.0
    bool correndo = tentandoCorrer && estaMovendo && !staminaEsgotada;

    if (correndo)
    {
        staminaAtual -= staminaGastoPorSegundo * Time.deltaTime;
        staminaAtual = Mathf.Clamp(staminaAtual, 0f, staminaMaxima);
    }
}

```

```

        timerRecuperacao = 0f;
        podeRecuperar = false;

        if (staminaAtual <= 0f)
            staminaEsgotada = true;
    }
    else
    {
        if (!podeRecuperar)
        {
            timerRecuperacao += Time.deltaTime;
            if (timerRecuperacao >= delayParaRecuperar)
                podeRecuperar = true;
        }

        if (podeRecuperar && staminaAtual < staminaMaxima)
        {
            staminaAtual += staminaRecuperacaoPorSegundo * Time.deltaTime;
            staminaAtual = Mathf.Clamp(staminaAtual, 0f, staminaMaxima);
        }

        if (staminaEsgotada && staminaAtual >= staminaMaxima * 0.3f)
            staminaEsgotada = false;
    }
}

private void AtualizarUI()
{
    if (barraStamina == null) return;

    barraStamina.value = staminaAtual;

    if (preenchimentoStamina != null)
    {
        float proporcao = staminaAtual / staminaMaxima;
        preenchimentoStamina.color = Color.Lerp(corStaminaBaixa, corStaminaAlta, proporcao);
    }
}

public float StaminaAtual => staminaAtual;
public float StaminaMaxima => staminaMaxima;

public bool EstaCorrendo()
{
    bool tentando = Input.GetKey(KeyCode.LeftShift) || Input.GetKey(KeyCode.RightShift);
    float h = Input.GetAxis("Horizontal");
    float v = Input.GetAxis("Vertical");
    return tentando && (h > 0 || v > 0);
}

```

```

        bool movendo = new Vector2(h, v).sqrMagnitude > 0.01f;
        return tentando && movendo && !staminaEsgotada;
    }
}

```

A.3 EnemyPatrol.cs

C#

```

using UnityEngine;

public class EnemyPatrol : MonoBehaviour
{
    public Transform[] waypoints;
    public float speed = 3f;
    public float waitTime = 2f;

    private int currentWaypoint = 0;
    private float waitCounter;
    private bool isWaiting;

    void Update()
    {
        if (waypoints.Length == 0) return;

        if (isWaiting)
        {
            waitCounter -= Time.deltaTime;

            if (waitCounter <= 0)
            {
                isWaiting = false;
                NextWaypoint();
            }
            return;
        }

        Transform target = waypoints[currentWaypoint];

        Vector3 direction = (target.position - transform.position).normalized;
        transform.position += direction * speed * Time.deltaTime;

        if (direction != Vector3.zero)
        {
            transform.rotation = Quaternion.LookRotation(direction);
        }
    }
}

```

```

        if (Vector3.Distance(transform.position, target.position) < 0.2f)
        {
            isWaiting = true;
            waitCounter = waitTime;
        }
    }

    void NextWaypoint()
    {
        currentWaypoint = (currentWaypoint + 1) % waypoints.Length;
    }
}

```

A.4 EnemyVision.cs

C#

```

using UnityEngine;

public class EnemyVision : MonoBehaviour
{
    public float viewDistance = 10f;
    [Range(0, 180)]
    public float viewAngle = 90f;
    public LayerMask targetMask;
    public LayerMask obstacleMask;

    [HideInInspector]
    public Transform target;
    [HideInInspector]
    public bool playerInSight;

    void Update()
    {
        FindTarget();
    }

    void FindTarget()
    {
        playerInSight = false;
        target = null;

        Collider[] targetsInViewRadius = Physics.OverlapSphere(transform.position, viewDistance, targetMask, ~obstacleMask);

        for (int i = 0; i < targetsInViewRadius.Length; i++)
        {
            if (targetsInViewRadius[i].gameObject.CompareTag("Player"))
            {
                playerInSight = true;
                target = targetsInViewRadius[i].transform;
                break;
            }
        }
    }
}

```

```

    {
        Transform potentialTarget = targetsInViewRadius[i].transform;
        Vector3 dirToTarget = (potentialTarget.position - transform.position).normalized;

        if (Vector3.Angle(transform.forward, dirToTarget) < viewAngle / 2)
        {
            float dstToTarget = Vector3.Distance(transform.position, potentialTarget.position);

            if (!Physics.Raycast(transform.position, dirToTarget, dstToTarget))
            {
                target = potentialTarget;
                playerInSight = true;
                break;
            }
        }
    }
}

void OnDrawGizmos()
{
    Vector3 origin = transform.position;
    Vector3 forward = transform.forward * viewDistance;

    Quaternion leftRotation = Quaternion.Euler(0f, -viewAngle / 2f, 0f);
    Quaternion rightRotation = Quaternion.Euler(0f, viewAngle / 2f, 0f);

    // Cone de visão
    Gizmos.color = Color.red;
    Gizmos.DrawRay(origin, leftRotation * forward);
    Gizmos.DrawRay(origin, rightRotation * forward);

    // Raio máximo de detecção
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(origin, viewDistance);

    // Linha até o alvo com cor por estado
    if (target != null)
    {
        Gizmos.color = playerInSight ? Color.green : Color.white;
        Gizmos.DrawLine(origin, target.position);
    }
}
}

```

A.5 EnemyDetection.cs

C#

```

using UnityEngine;

public class EnemyDetection : MonoBehaviour
{
    [Header("Referências")]
    public EnemyVision enemyVision;
    public EnemyPatrol enemyPatrol;
    public Transform target;

    [Header("Configuração de alerta")]
    public float rotationSpeed = 5f;

    private bool isAlert = false;

    void Update()
    {
        if (enemyVision.playerInSight)
        {
            target = enemyVision.target;
            if (!isAlert)
            {
                isAlert = true;
                enemyPatrol.enabled = false; // Para a patrulha
                Debug.Log("Jogador detectado! Entrando em alerta.");
            }
            LookAtTarget();
        }
        else
        {
            if (isAlert)
            {
                isAlert = false;
                enemyPatrol.enabled = true; // Retoma a patrulha
                Debug.Log("Jogador perdido de vista. Retomando patrulha.");
            }
        }
    }

    void LookAtTarget()
    {
        if (target != null)
        {
            Vector3 directionToTarget = (target.position - transform.position).normalized;
            Quaternion lookRotation = Quaternion.LookRotation(new Vector3(directionToTarget.x, directionToTarget.y, 0f));
            transform.rotation = Quaternion.Slerp(transform.rotation, lookRotation, rotationSpeed * Time.deltaTime);
        }
    }
}

```



```
public bool IsAlert => isAlert;  
}
```